

Optimization of a Voltage-Source Inverter Control System Using Evolutionary Strategies and Differential Evolution

Antoni Kingston^{1*} and Marcel Wronkowski^{1†}

^{1*}Faculty Mathematics and Information Sciences, Warsaw University of Technology,.

[†]These authors contributed equally to this work.

Abstract

Tuning discrete-time DLQR weights for a voltage-source inverter is posed as a four-dimensional black-box minimization problem. A MATLAB simulator returns a scalar cost from closed-loop time-domain evaluation; unstable designs are penalized. We compare (μ, λ) evolution strategy, self-adaptive differential evolution (jDE), covariance-matrix adaptation evolution strategy (CMA-ES), and a native particle-swarm optimization baseline under matched function-evaluation budgets $B \in \{50, 100, 200\}$, with reproducible batch runs and non-parametric tests (Mann–Whitney with Holm correction, Wilcoxon, Friedman) on configurations tuned at $B = 50$. Descriptively, jDE and CMA-ES reach fitness near **0.018**, whereas PSO remains near **0.15** and does not improve when B increases. jDE is the most stable across seeds; CMA-ES requires $B \geq 100$ to match jDE on tuned settings; evolution strategy runs show occasional severe outliers. With only five seeds per tuned configuration, Holm-corrected tests support jDE and CMA-ES over PSO at $B = 100$ and $B = 200$, but not evolution strategy versus PSO, and no test confirms a benefit from larger B . We recommend jDE for robustness or CMA-ES when wall-clock cost matters at moderate budgets, and treat PSO as a fast but suboptimal reference on this simulator.

Keywords: evolutionary computation, evolution strategies, differential evolution, CMA-ES, particle swarm optimization, voltage-source inverter, black-box optimization

1 Introduction

Grid-connected voltage-source inverters (VSIs) with LC output filters must provide accurate reference tracking while limiting control effort and respecting actuator limits. A common design approach combines a state-feedback regulator with weighting matrices Q and R that trade off error suppression against input activity. For the discrete-time DLQR structure used in our simulator, these weights are not chosen by hand: they are obtained from a candidate parameter vector that defines diagonal penalties on plant, input, and resonant states before each closed-loop run. Because each trial requires solving an algebraic Riccati equation and simulating the nonlinearities introduced by saturation and instability checks, the mapping from parameters to performance is expensive, non-smooth, and not available in closed form.

Derivative-free optimizers are a natural match for such settings. Evolution strategies and differential evolution explore the search space through populations and mutations without gradients; covariance-matrix adaptation evolution strategy (CMA-ES) models variable interactions through an adapting Gaussian distribution; particle swarm optimization (PSO) provides a widely used engineering baseline. The WdAE project task examined exactly this scenario: tune a four-dimensional weight vector for a supplied VSI simulator, compare evolutionary methods with PSO, and report reproducible numerical results with appropriate statistical tests.

In this work we implement and benchmark three Python-based optimizers—a (μ, λ) evolution strategy with self-adapted step size, jDE (DE/rand/1/bin with adaptive F and CR), and CMA-ES via `pycma`—against the MATLAB PSO routine bundled with the simulator. All methods minimize the same scalar fitness $f(\mathbf{p})$ (Section 2) under explicit function-evaluation budgets $B \in \{50, 100, 200\}$. Python algorithms query fitness through an XML-RPC server; PSO evaluates f natively inside MATLAB. Hyperparameters are explored on coarse grids; representative configurations are selected on the smallest budget and analysed across all B . Non-parametric tests (Mann–Whitney with Holm correction, paired Wilcoxon tests, Friedman test on matched seeds) are applied with $n = 5$ seeds per tuned setting, and outcomes are reported without overstating significance where sample size limits power.

The study addresses three practical questions:

1. Can evolutionary methods consistently reach lower fitness than the PSO baseline under equal evaluation budgets?
2. Among ES, jDE, and CMA-ES, which method offers the best trade-off between final fitness, robustness across seeds, and wall-clock time?
3. Does increasing B from 50 to 200 still improve controller quality once a good weight vector has been found?

Our contributions are:

- a unified experimental harness (`run.experiments.py`) with CSV logging, optional trajectories, and parallel execution for non-PSO methods;
- corrected integration of PSO hyperparameters and random seeds through `PSOMatlabConfig` and `main.m`;

- a full grid study on the provided simulator with analysis scripts for figures and statistics;
- an honest discussion of where differences are statistically supported versus merely descriptive (Sections 5–7).

The remainder of the report is organized as follows. Section 2 formalizes the simulator, decision variables, and cost function. Section 3 describes each optimizer and its coupling to the evaluation budget. Section 4 documents grids, seeds, and testing procedures. Section 5 presents fitness distributions, tuned-configuration tables, and runtime behaviour. Sections 6 and 7 interpret the findings and state recommendations. Section 8 records differences from the preliminary WdAE project plan. Reproduction instructions are given in the public repository <https://github.com/AntoniKingston/VSI-Circuit-Optimization-Utilizing-Evolutionary-Strategies> (README.md).

2 Problem formulation

We study the task of tuning a discrete-time DLQR controller for a three-phase voltage-source inverter (VSI) with an LC output filter. The plant, reference generator, resonant augmentation, and closed-loop simulation are provided as a MATLAB black-box evaluator. The optimizer does not access model Jacobians or cost gradients; it may only query a scalar fitness for candidate weighting vectors.

2.1 Plant, augmentation, and DLQR control

The averaged VSI dynamics are modelled in the $\alpha\beta$ frame as a linear continuous-time state-space system with states associated with filter inductor currents and capacitor voltages, and two control inputs. The model is discretized by zero-order hold at sampling period $T_s = 1/(2f_{\text{sw}})$, where f_{sw} is the switching frequency. Input delay is represented by augmenting the state with previous actuation values, and harmonic tracking is introduced through resonant states driven by the tracking error (one harmonic at grid frequency in the default configuration).

For a candidate parameter vector $\mathbf{p} \in \mathbb{R}^4$, the simulator constructs diagonal weighting matrices $Q(\mathbf{p})$ and $R(\mathbf{p})$ and solves the discrete-time algebraic Riccati equation to obtain a stabilizing gain matrix $K(\mathbf{p})$. The gain is decomposed into components (K_x, K_u, K_r) acting on the plant state, delayed input, and resonant states, respectively. At each discrete time step k , the controller computes

$$\mathbf{u}_k = -K_r \mathbf{x}_{r,k} - K_u \mathbf{x}_{u,k} - K_x \mathbf{x}_k, \quad (1)$$

with actuation saturated to the DC-link limits $\pm V_{\text{dc}}$. The closed-loop state is advanced using the augmented discrete matrices (A_d, B_d) supplied by the simulator configuration (filter parameters L_f, C_f, R_f , bus voltage, simulation horizon, and penalty weight β_c).

Before simulation, the spectral radius of the closed-loop matrix $A_{\text{cl}} = A_{d,\text{aug}} - B_{d,\text{aug}}K$ is checked. If $\rho(A_{\text{cl}}) \geq 1$, the candidate is rejected with a large penalty fitness (numerically 10^6), as it corresponds to an unstable or marginally unstable discrete-time loop.

2.2 Decision variables and feasible set

The optimization vector $\mathbf{p} = (p_1, \dots, p_4)^\top$ encodes DLQR weights in *logarithmic design coordinates*. Each component is mapped to positive diagonal entries of Q through 10^{p_j} , following the structure implemented in `calculate_k_matrix.m`: two components govern state penalties on the tracked $\alpha\beta$ output, one component scales control-input penalties, and one component sets the weight of the resonant (harmonic) states. With `R_mode = 0` in the default simulator setup, the input-cost matrix R is fixed and not part of the search; only Q is tuned.

In the evolutionary and CMA-ES experiments, each p_j is constrained to the box

$$p_j \in [-10, 10], \quad j = 1, \dots, 4, \quad (2)$$

and candidate vectors are clipped to this set before evaluation. The particle-swarm baseline uses the native bound generation of the MATLAB optimizer (component-wise limits derived from `Qmax` in the simulator parameters, typically $[-4, 4]$ per dimension), which defines a slightly different feasible set; algorithm comparisons therefore refer to the same fitness oracle but not necessarily to identical box constraints across methods.

2.3 Objective function

For a stabilizing parameter vector $\mathbf{p}$, fitness is the time-average quadratic cost computed over a fixed horizon T_{sim} with $N = \lfloor T_{\text{sim}}/T_s \rfloor$ steps. Let $\mathbf{e}_k \in \mathbb{R}^{n_u}$ denote the tracking error between the reference and the selected outputs at step k , and $\Delta \mathbf{u}_k$ the increment of the (saturated) control input. The simulator returns

$$f(\mathbf{p}) = \frac{1}{N} \sum_{k=1}^N (\|\mathbf{e}_k\|_2^2 + \beta_c \|\Delta \mathbf{u}_k\|_2^2), \quad (3)$$

where $\beta_c > 0$ penalizes aggressive control moves. The reference is a smooth step in the $\alpha\beta$ frame with amplitude V_{ref} and settling time constant derived from t_{settling} . Lower fitness values indicate better tracking with moderate control effort.

Failed numerical operations (e.g. DLQR design failure or simulation exception) and unstable closed loops are mapped to the same large penalty value, which shapes the feasible landscape seen by the optimizers without explicit constraint handling beyond clipping in $\mathbf{p}$.

2.4 Black-box optimization problem

The engineering goal is to minimize fitness subject to implicit stability and simulator-side validity:

$$\mathbf{p}^* \in \arg \min_{\mathbf{p} \in \mathcal{P}} f(\mathbf{p}), \quad (4)$$

where $\mathcal{P} \subset \mathbb{R}^4$ is the box in (2) (or the PSO-specific bounds for the baseline). The mapping $\mathbf{p} \mapsto f(\mathbf{p})$ is deterministic for a fixed random seed in the stochastic optimizers, but it is nonconvex, non-smooth in practice because of penalties and saturation,

and expensive: each evaluation requires building (Q, R) , solving DLQR, and running the full time-domain simulation.

Algorithms are compared under a common *function evaluation budget* (FES): one evaluation equals one call to the XML-RPC `evaluate` interface (Python `client.py` forwarding $\mathbf{p}$ to the MATLAB server on 127.0.0.1:8484), which internally executes the pipeline in `DLQREvaluator.evaluate`. This ensures that ES, jDE, CMA-ES, and PSO incur comparable oracle costs even though their internal iteration structure differs.

2.5 Modelling assumptions relevant to the study

The following assumptions follow directly from the provided simulator and from the experimental protocol used later in this report:

- **No analytical gradient.** Optimizers treat f as a black box; no adjoint or sensitivity model is available.
- **No certified global optimum.** The best achievable fitness is unknown a priori; improvements are assessed empirically across algorithms and seeds.
- **Deterministic oracle.** For a given $\mathbf{p}$ and fixed simulator data, $f(\mathbf{p})$ is reproducible; stochasticity enters only through the search algorithms (and optional RNG seeding in PSO).
- **Penalty-dominated failures.** Unstable or infeasible designs are not distinguished by separate constraint violations; they collapse to the same scalar penalty, which can create flat or deceptive regions in the search space.
- **Fixed plant and scenario.** Filter parameters, horizon, reference profile, and β_c are held constant; the study isolates controller tuning rather than plant identification or robustness to parameter uncertainty.

Together, (4)–(3) define the optimization problem addressed by evolution strategies, adaptive differential evolution, CMA-ES, and the PSO reference implementation in the remainder of this work.

3 Compared algorithms

We compare four derivative-free optimizers on the problem formulated in Section 2: a (μ, λ) evolution strategy (ES) with self-adapted global step size, the *jDE* variant of differential evolution, covariance-matrix adaptation evolution strategy (CMA-ES), and a particle-swarm optimization (PSO) routine supplied with the simulator. The first three are implemented in Python and query fitness through an XML-RPC client; PSO runs natively in MATLAB and calls the evaluator object directly. All methods search the same four-dimensional weight space $\mathbf{p}$, but they differ in population structure, adaptation mechanisms, and how a fixed evaluation budget B is translated into generations.

3.1 Common experimental interface

Let B denote the FES allocated to a single run. For ES, jDE, and CMA-ES, one function evaluation is one invocation of `client.py`, which forwards $\mathbf{p}$ to the simulator server and returns $f(\mathbf{p})$. Candidate vectors are clipped to $[-10, 10]^4$ before evaluation (Section 2.2). Python optimizers receive an independent pseudorandom seed per run; PSO additionally fixes the MATLAB RNG when a non-negative seed is passed to `main.m`.

Table 1 summarizes how B is mapped to iterations and which hyperparameters are tuned in the grid search (Section 4). The effective number of evaluations may deviate slightly from B for ES because of initialization logging, but the design intent is to align methods to comparable oracle cost.

Table 1 Algorithm families, tuned hyperparameters, and budget mapping for one run with FES B .

Method	Tuned settings	Generations / iterations	Evaluations per cycle
(μ, λ) -ES	$\mu, \lambda, \sigma_0, \sigma_\sigma$	$G = \lfloor B/\lambda \rfloor$	λ offspring
jDE	$N_{\text{pop}}, F_0, \text{CR}_0, \tau$	$G = \lfloor B/N_{\text{pop}} \rfloor$	$\leq N_{\text{pop}}$ trials
CMA-ES	$\sigma_0, \text{popsize}$	until B fevals	one population
PSO	$N_{\text{part}}, c_1, c_2, \rho_{\text{rang}}$	$G = \lfloor B/N_{\text{part}} \rfloor$	N_{part} particles

3.2 (μ, λ) evolution strategy

General method.

Evolution strategies are stochastic search methods designed for continuous black-box optimization in $\mathbb{R}^n$. In the (μ, λ) selection scheme, μ parent individuals produce λ offspring per generation ($\lambda \geq \mu$); only the offspring compete for survival, and the best μ become parents of the next generation. Mutation is typically applied in an isotropic or correlated manner; the step-size control parameter σ is often self-adapted so that the search scale matches the local fitness landscape.

Implementation and encoding.

Our implementation follows the (μ, λ) pattern with intermediate recombination and Gaussian mutation of the decision vector. Each individual stores the four weights $\mathbf{p}$ plus one strategy parameter σ appended to the genotype. Fitness calls use only $\mathbf{p}$; σ governs the magnitude of mutation and evolves by multiplicative log-normal perturbation with learning rate σ_σ :

$$\sigma \leftarrow \sigma \cdot \exp(\mathcal{N}(0, \sigma_\sigma^2)), \quad (5)$$

with clipping to $[10^{-8}, 10]$ for numerical stability. Offspring are generated by:

1. selecting two distinct parents uniformly at random;

2. applying one-point crossover with probability 0.9 (otherwise cloning one parent);
3. adding $\mathcal{N}(\mathbf{0}, \sigma^2 I_4)$ noise to $\mathbf{p}$ and adapting σ as in (5);
4. clipping $\mathbf{p}$ to the feasible box.

The μ best offspring under f form the next parent population.

Application to the VSI problem.

For budget B , the runner sets $G = \max(1, \lfloor B/\lambda \rfloor)$ generations. Hyperparameters μ , λ , initial step size σ_0 , and σ_σ are swept on a grid; each combination is run with multiple random seeds. Because unstable DLQR designs yield the flat penalty 10^6 , σ adaptation can occasionally drive the strategy into aggressive mutations; clipping and parent-offspring selection mitigate but do not eliminate this risk. The method is well suited to moderate dimensionality ($n = 4$) but has no explicit model of variable coupling beyond the diagonal Gaussian mutation.

3.3 Adaptive differential evolution (jDE)

General method.

Differential evolution (DE) maintains a population of N_{pop} candidate solutions and creates trials by combining scaled differences between existing members. The `rand/1/bin` variant draws three distinct indices r_1, r_2, r_3 and forms a mutant

$$\mathbf{v} = \mathbf{x}_{r_1} + F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3}), \quad (6)$$

then performs binomial crossover with crossover rate CR between $\mathbf{v}$ and the target $\mathbf{x}_i$, ensuring at least one component comes from the mutant. Greedy selection replaces $\mathbf{x}_i$ if the trial is not worse.

jDE [1] extends DE by encoding F and CR in each individual and resetting them probabilistically each generation: with probability τ (per parameter), F or CR is re-sampled uniformly from fixed intervals (here $F \in [0.1, 1]$ and $\text{CR} \in [0, 1]$). This removes the need to hand-tune a single (F, CR) pair across the VSI landscape.

Implementation and encoding.

Each population member stores $(\mathbf{p}, F, \text{CR})$. For every target index i , the algorithm adapts (F, CR) , builds $\mathbf{v}$ via (6), applies binomial crossover, clips the trial to $[-10, 10]^4$, and compares $f(\mathbf{p}_{\text{trial}})$ with $f(\mathbf{p}_i)$. The population is initialized uniformly in the box; F and CR are initialized to grid values F_0 and CR_0 with adaptation intensity τ (shared for both parameters in our code).

Application to the VSI problem.

With population size N_{pop} , one generation performs at most N_{pop} new evaluations (fewer if trials are rejected before calling the oracle only when selection succeeds—in our implementation each accepted comparison requires one trial evaluation). The outer loop runs $G = \lfloor B/N_{\text{pop}} \rfloor$ generations for budget B . jDE is attractive when the

fitness landscape is multimodal: difference vectors can jump between basins in log-weight space. On the VSI task, penalty regions around unstable gains act as barriers; self-adaptive F and CR moderate the step size when progress stalls.

3.4 Covariance-matrix adaptation ES (CMA-ES)

General method.

CMA-ES [2] adapts a multivariate normal search distribution $\mathcal{N}(\mathbf{m}, \sigma^2 C)$ over generations. The mean $\mathbf{m}$ is updated by rank-weighted recombination of the best offspring; the step size σ is controlled by cumulative step-length adaptation; and the covariance matrix C captures variable interactions and scaling. The method is invariant to affine transformations of the search space (up to the initial σ_0 and population size) and is widely used on ill-conditioned black-box problems.

Implementation.

We use the `pycma` reference implementation with box constraints enforced by passing lower and upper bounds $[-10, 10]^4$. The initial mean is the centre of the box; initial step size σ_0 and population size `popsize` are configuration inputs (not evolved). The evolutionary loop follows the standard `ask--tell` pattern: sample λ points, evaluate f , update the internal state, and stop when the evaluation count reaches B (`maxfevals=B`). The random seed is passed into CMA-ES options for reproducibility.

Application to the VSI problem.

CMA-ES is the only compared method that explicitly models correlations between log-weights, which may matter when state, input, and harmonic penalties interact through the closed loop. Because $n = 4$, population sizes `popsize` $\in \{8, 16\}$ from the experimental grid are above the default minimum recommended by the theory, but still small compared to large-scale benchmark studies. The method does not use penalty-aware repair beyond bound clipping; unstable $\mathbf{p}$ still consume budget through 10^6 evaluations. In practice CMA-ES often achieves low fitness with fewer effective evaluations than ES on similar black-box tasks, at the cost of higher per-iteration linear algebra overhead (negligible relative to the MATLAB simulation).

3.5 Particle swarm optimization (baseline)

General method.

PSO maintains a swarm of N_{part} particles with positions $\mathbf{x}_i$ and velocities $\mathbf{v}_i$ in the search space. Each particle remembers its personal best $\mathbf{p}_i$ and is attracted toward the global best $\mathbf{g}$ and its own history. With constriction-factor coefficients c_1 , c_2 , the velocity update uses

$$\mathbf{v}_i \leftarrow \kappa \left(\mathbf{v}_i + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i) \right), \quad (7)$$

where κ is derived from $c_1 + c_2$ for stability, and $r_1, r_2 \sim \mathcal{U}(0, 1)$. Positions are updated and projected onto box constraints (absorbing walls zero velocity on boundary contact).

Implementation.

The baseline `PsoOptimizer` class in `src/matlab` evaluates $f(\mathbf{p})$ through `DLQREvaluator.evaluate` inside a serial loop over particles (no parallel pool). Hyperparameters c_1 , c_2 , and ρ_{rang} (field `rang_coef`) control cognitive/social attraction and the span of initial positions around the search-space centre. Function `main.m` sets the iteration count to $G = \lfloor B/N_{\text{part}} \rfloor$, fixes the RNG seed per run, and returns the best fitness via `PSO_FINAL_FITNESS` in the log (also exposed as a MATLAB return value).

Application to the VSI problem.

PSO serves as an engineering baseline shipped with the simulator rather than as a peer method implemented in the Python stack. Initial particle positions are drawn in a reduced box $[-4, 4]^4$ (from `Qmax`), which is tighter than the $[-10, 10]^4$ clipping used by the evolutionary methods—a methodological asymmetry to remember when interpreting comparisons. Each swarm iteration evaluates every particle once, so $B \approx G \cdot N_{\text{part}}$. PSO does not adapt step-size statistics to the fitness landscape beyond velocity updates; on the VSI task it historically converges to markedly worse fitness than ES, jDE, or CMA-ES at the same budget, but at lower wall-clock cost because it avoids the XML-RPC subprocess overhead.

3.6 Summary of methodological choices

All evolutionary methods treat f as a minimization objective without explicit constraint handling beyond clipping and the simulator’s stability check. They differ in exploration mechanism: ES mutates around elite parents with a scalar σ , jDE uses population differences with adaptive (F, CR) , CMA-ES learns a full covariance structure, and PSO performs velocity-based social search. For fair reporting, results in Section 5 are organized by fixed B , tuned configuration, and seed; statistical tests operate on the resulting distributions of best fitness and runtime.

4 Experimental methodology

Experiments used FES budgets $B \in \{50, 100, 200\}$, seeds $\{1, \dots, 5\}$, and the hyperparameter grids in `experiments_config.txt` (ES: $\mu \in \{10, 20, 40\}$, $\lambda \in \{20, 50, 100\}$; jDE: population size $\in \{10, 20, 40\}$; CMA-ES: popsize $\in \{8, 16\}$; PSO: $N_{\text{part}} \in \{5, 15, 30\}$, etc.). Each algorithm–configuration–budget–seed tuple produced one row in `results/detailed_results.csv`. For reporting, configurations were tuned by lowest median fitness on $B = 50$ and evaluated at all budgets (Section 5.2). Statistical analysis (`statistical_tests.py`) comprised Mann–Whitney pairwise comparisons with Holm correction, paired Wilcoxon budget tests, Friedman tests on matched seeds, and Cliff’s δ effect sizes. PSO ran serially (`parallel_pso=0`) on a shared MATLAB engine; other methods used parallel workers where configured.

Table 2 Pooled median best fitness by algorithm and FES budget B (all grid points, successful runs only). Descriptive summary; n varies per cell (hundreds of runs per algorithm).

Algorithm	$B = 50$	$B = 100$	$B = 200$
ES	0.0250	0.0226	0.0190
jDE	0.0183	0.0183	0.0183
CMA-ES	0.355	0.0186	0.0183
PSO	0.309	0.153	0.149

5 Results

Experiments were executed with `run_experiments.py` on the full hyperparameter grids of Section 4, FES budgets $B \in \{50, 100, 200\}$, and five random seeds per configuration. The run archive contains 3330 detailed records in `results/detailed_results.csv` (success rate above 98% for ES/jDE/PSO; all CMA-ES runs completed). Below we report outcomes most relevant to the comparisons in Sections 6 and 7; additional sensitivity plots are available under `results/figures/`.

5.1 Overall fitness levels

Table 2 lists *descriptive* pooled medians over all grid configurations at each budget (not an inferential sample; configurations and seeds are pooled). Evolutionary methods often reach fitness of order 10^{-2} , while PSO remains near 0.15–0.31. At $B = 200$, jDE and CMA-ES attain median 0.0183; ES is slightly higher (0.0190) because of rare catastrophic outliers (Section 5.4). At $B = 50$, the pooled CMA-ES median (0.36) is even worse than PSO (0.31), so a small budget is not sufficient for reliable CMA-ES performance across the whole grid.

Figure 1 shows full distributions at the largest budget. Visually, many evolutionary runs cluster near 0.018, whereas PSO occupies a separate band above 0.14; this is a graphical pattern, not by itself a hypothesis test on tuned configurations.

5.2 Tuned configurations

For a per-method comparison, we selected the best hyperparameter set on the tuning budget $B = 50$ (lowest median fitness over five seeds) and re-evaluated it at $B \in \{50, 100, 200\}$. Table 3 summarizes these runs ($n = 5$ seeds per cell—the inferential unit for the tests below). jDE is the most stable: median fitness stays at 0.01826 across budgets. CMA-ES recovers after tuning once $B \geq 100$. The selected ES configuration performs well at $B = 50$ but *degrades* when the budget increases (median 0.047 at $B = 200$). PSO under its tuned settings plateaus at 0.149 independently of B .

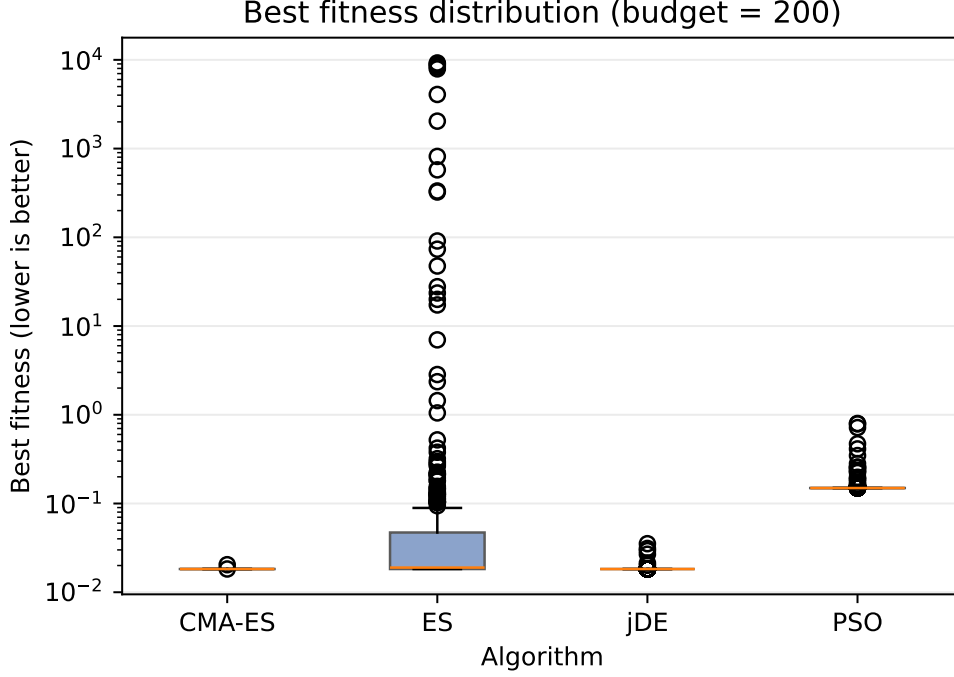


Fig. 1 Distribution of best fitness at $B = 200$ over all hyperparameter configurations (log scale).

Table 3 Median best fitness for configurations tuned on $B = 50$ and tested at each budget ($n = 5$ seeds per cell).

Algorithm	$B = 50$	$B = 100$	$B = 200$
ES	0.0183	0.0304	0.0466
jDE	0.0183	0.0183	0.0183
CMA-ES	0.0302	0.0183	0.0183
PSO	0.149	0.149	0.149

Statistical tests ($n = 5$, tuned configurations).

Table 4 reports Mann–Whitney U tests (two-sided) with Holm-adjusted p -values on the tuned series; this is the analysis aligned with our pre-registered plan, but with only five seeds per comparison the power is limited. At $B = 200$: jDE and CMA-ES differ significantly from PSO ($p_{\text{Holm}} \approx 0.022$), and jDE/CMA-ES differ significantly from ES ($p_{\text{Holm}} \approx 0.045$) because ES medians are higher under the selected config. *However*,

Table 4 Selected Mann–Whitney tests on tuned configurations ($n = 5$ per group). Holm correction applied within each budget row. Significant at $\alpha = 0.05$ shown in bold.

B	Comparison	p (raw)	p_{Holm}	Sig.
50	jDE vs PSO	0.011	0.067	
50	CMA-ES vs PSO	0.139	0.556	
50	ES vs PSO	0.672	1.000	
100	jDE vs PSO	0.007	0.045	yes
100	CMA-ES vs PSO	0.007	0.045	yes
100	ES vs PSO	0.119	0.119	
200	jDE vs PSO	0.007	0.045	yes
200	CMA-ES vs PSO	0.007	0.045	yes
200	ES vs PSO	0.119	0.119	
200	jDE vs ES	0.008	0.045	yes

ES versus PSO at $B = 200$ is not significant ($p = 0.119$, $p_{\text{Holm}} = 0.119$). At $B = 50$, no pairwise comparison survives Holm correction at $\alpha = 0.05$ (the smallest raw p is jDE vs PSO, 0.011, but $p_{\text{Holm}} = 0.067$). Friedman tests on matched seeds across the four tuned algorithms are non-significant at $B = 50$ ($p = 0.11$) but significant at $B = 100$ and $B = 200$ ($p \approx 0.003$), indicating global differences only at larger budgets. Paired Wilcoxon tests for budget scaling ($50 \rightarrow 100 \rightarrow 200$ on the same seeds) are *not* significant for any algorithm ($p \geq 0.06$): we cannot claim that increasing B improves tuned fitness in a statistically confirmed way. We deliberately do *not* treat pooled grid-wide samples as confirmatory evidence: mixing hundreds of configurations inflates n and yields artificially small p -values unrelated to the tuned-comparison design.

5.3 Budget scaling

Figure 2 plots median–IQR fitness versus B for the tuned configurations of Table 3. jDE and CMA-ES appear to stabilize near 0.018 as B grows; ES and PSO show flat or worsening medians under the chosen tuning protocol. Statistically, Wilcoxon signed-rank tests on matched seeds do not support a budget effect ($p \geq 0.06$ for all algorithms): the descriptive trends in the figure should not be read as confirmed improvements from larger B .

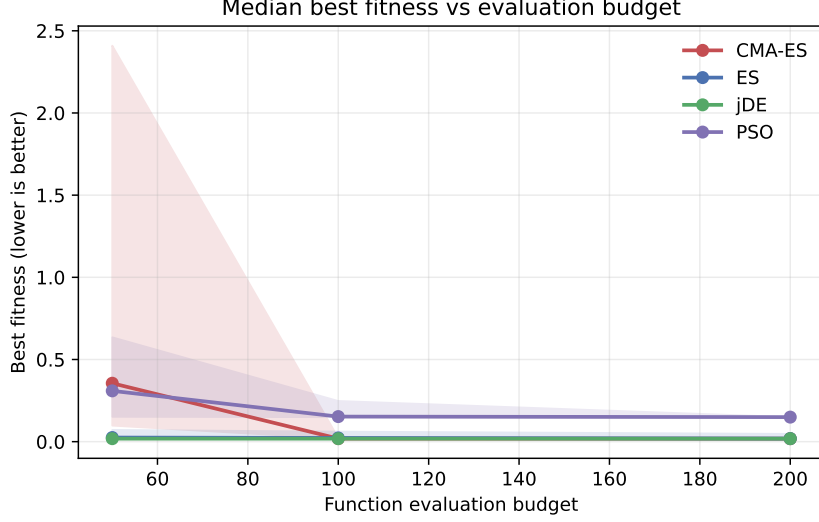


Fig. 2 Median and IQR of best fitness versus FES budget for tuned configurations.

5.4 Robustness and runtime

ES produced severe outliers (fitness $\gg 1$) in 6%–12% of successful runs depending on B , caused by unstable closed-loop trials penalized at 10^6 but occasionally returning large finite costs before failure. jDE and tuned CMA-ES runs stayed close to the optimum except for isolated seeds. PSO never reached the evolutionary fitness band but showed low variance around its plateau.

Wall-clock time (Figure 3) increases roughly linearly with B and is dominated by MATLAB simulation and XML-RPC overhead for Python methods. At $B = 200$, median runtimes are approximately 3.5 s (PSO), 61 s (CMA-ES), 67 s (ES), and 149 s (jDE).

6 Discussion

The experiments indicate that tuning DLQR weights for the provided VSI simulator is a low-dimensional but harsh black-box problem: a narrow fitness valley near $f \approx 0.01826$ coexist with a large penalty region and, for PSO, a separate local regime near $f \approx 0.15$.

Evolutionary methods versus PSO.

Descriptively, PSO under tuned settings stays near $f \approx 0.15$ while jDE and CMA-ES reach $f \approx 0.018$ (Table 3). Under the planned tests ($n = 5$), jDE and CMA-ES are significantly better than PSO at $B = 100$ and $B = 200$ after Holm correction, but **not** ES versus PSO at $B = 200$, and **no** comparison at $B = 50$ survives Holm correction. We therefore state that PSO is empirically far from the best evolutionary fitness values, but the statistical evidence for dominance is method-specific and budget-dependent rather than universal across all evolutionary algorithms. Pooled summaries (Table 2)

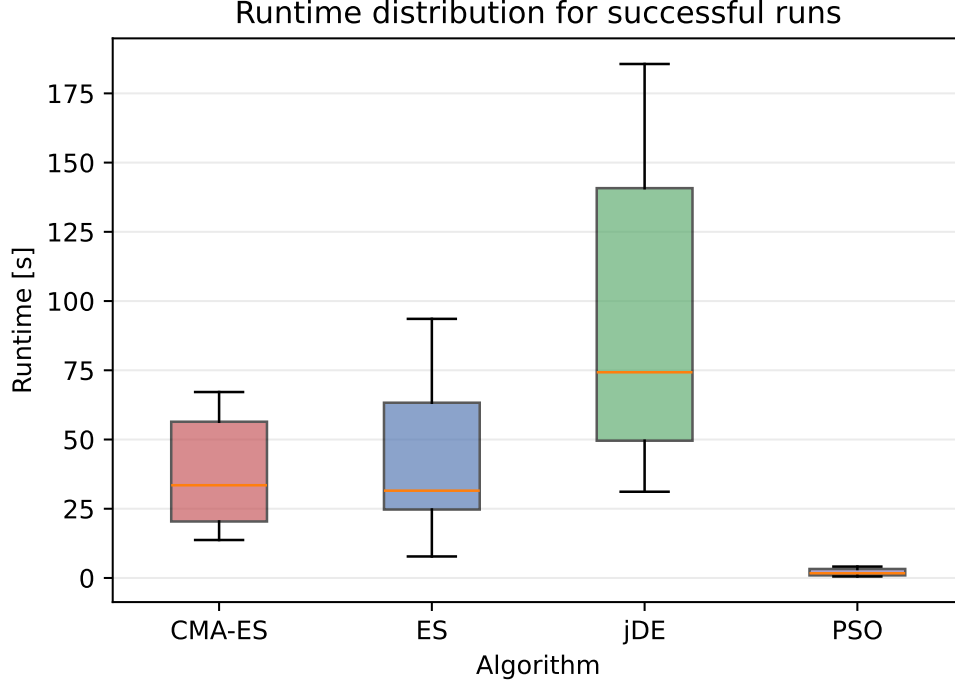


Fig. 3 Wall-clock runtime per successful run (log scale), grouped by algorithm and budget.

and Figure 1 illustrate the same separation graphically; they must not be quoted as confirmatory tests because they aggregate the full hyperparameter grid. Contributing factors include the tighter PSO search box $([-4, 4]^4$ versus $[-10, 10]^4$) and different integration paths (native MATLAB vs XML-RPC).

Comparison among ES, jDE, and CMA-ES.

Across the full grid, all three can reach fitness near 0.01826, but the *tuned* ES configuration selected on $B = 50$ is significantly worse than jDE and CMA-ES at $B = 100$ and $B = 200$ ($p_{\text{Holm}} \approx 0.045$). At $B = 200$, jDE and CMA-ES are also significantly different in the Mann–Whitney test ($p_{\text{Holm}} \approx 0.045$) despite nearly identical medians in Table 3—a consequence of very low seed-to-seed variance and $n = 5$. jDE shows the best *reliability* in Tables 3–4: stable medians and no significant budget effect. CMA-ES is weak on the full grid at $B = 50$ (Table 2) but matches jDE once tuned configs run at $B \geq 100$. ES still produces the worst tail behaviour in pooled runs (Section 5.4); oracle best-per-seed medians can match jDE, which shows that ES *can* find the valley, but not reliably under the tuned settings reported here.

Budget scaling and the fitness plateau.

Wilcoxon tests do not confirm that increasing B improves tuned fitness for any algorithm. Descriptively, jDE and CMA-ES medians are already near 0.01826 at

$B = 50$ – 200 , while ES medians rise with B under the selected config—a pattern that may reflect continued exploration of unstable weights rather than a validated budget benefit. We interpret the plateau as a property of the simulator landscape, but that interpretation rests on descriptive tables and non-significant Wilcoxon outcomes, not on rejected null hypotheses of improvement.

Computational trade-offs.

PSO is fastest (Figure 3) but sacrifices solution quality. Among evolutionary candidates, CMA-ES offers the best quality-per-second at $B = 200$; jDE is slowest because each generation performs up to N_{pop} sequential XML-RPC evaluations. The choice depends on whether wall-clock time or final fitness is binding.

Limitations.

Only five seeds per configuration were run, below the 30+ foreseen in the initial project plan, which limits statistical power for tuned-configuration tests. Results are conditional on the supplied simulator, fixed plant parameters, and grid choices; no claims are made about hardware-in-the-loop performance, THD, or overshoot metrics. Finally, configuration tuning used the lowest budget ($B = 50$), which favours methods that perform well with few evaluations (jDE) and may disadvantage CMA-ES unless results at larger B are inspected separately, as done here.

7 Conclusions

We optimized four DLQR weighting parameters of a voltage-source inverter simulator using (μ, λ) -ES, jDE, CMA-ES, and a native PSO baseline under shared function-evaluation budgets $B \in \{50, 100, 200\}$.

The main conclusions are:

1. **PSO is descriptively far from the best fitness, but statistical support is partial.** Tuned jDE and CMA-ES beat PSO significantly at $B = 100$ and $B = 200$ ($n = 5$, Holm-corrected); ES does not; at $B = 50$ no Holm-significant pairwise result was obtained.
2. **jDE is the most dependable tuned configuration.** It maintains median ≈ 0.01826 across budgets with low spread; Wilcoxon tests show no confirmed improvement when B increases.
3. **CMA-ES works well only after sufficient budget on the tuned config.** It matches jDE at $B \geq 100$ in Table 3, but pooled results at $B = 50$ remain poor.
4. **ES is unreliable under the tuned settings reported.** It is significantly worse than jDE/CMA-ES at larger B and exhibits heavy outliers in grid-wide runs.
5. **Larger B did not yield statistically confirmed gains** on tuned configs (Wilcoxon $p \geq 0.06$), although descriptive medians already sit near the apparent simulator floor.

Practical recommendation.

Under the tested sample size, jDE or CMA-ES at $B \geq 100$ are the methods with significant evidence of beating tuned PSO; jDE is preferable when stability across

Table 5 Planned versus realized elements of the study (initial WdAE report vs. final work).

Topic	Initial plan	Final study
FES budgets B	1000, 2000, 5000	50, 100, 200
Seeds per configuration	≥ 30	5
Algorithms	ES, jDE, PSO	ES, jDE, CMA-ES, PSO
CMA-ES	not listed	added (pycma)
PSO	repo baseline	repaired MATLAB bridge; optional rerun/merge scripts
Extra metrics	THD, overshoot	not done; cost f only
Statistics	Wilcoxon / MW + Holm	<code>statistical_tests.py</code> ; low power
Convergence	best-so-far vs FES	trajectories + optional plots
Experiment driver	—	<code>run_experiments.py</code> , parallel pool

seeds matters. ES and PSO should not be presented as statistically equivalent to jDE/CMA-ES on fitness, but ES versus PSO was not significantly different here. Increasing seeds and rerunning the Holm pipeline would be needed for stronger claims. Future work should increase the number of independent seeds, log best-so-far trajectories uniformly across methods, and validate winning controllers with time-domain performance indicators beyond the scalar cost (3).

8 Deviations from the initial report

The preliminary WdAE document (task 24, VSI + DLQR tuning) defined the research direction: black-box minimization of simulator fitness with evolution strategies, jDE, comparison to PSO, reproducible CSV outputs, and non-parametric statistical testing. The final deliverable follows that core idea but differs in several concrete choices driven by integration effort, runtime limits, and what could be completed before the submission deadline. Table 5 summarizes the main gaps; the paragraphs below explain rationale and impact on interpretation.

8.1 Scope and algorithms

The problem formulation (four log-weights, DLQR + time-domain simulation, penalty for instability) is unchanged from the initial proposal. **CMA-ES was added** as a modern continuous optimizer with covariance adaptation, which was not named in the preliminary document but fits the comparison of derivative-free methods on the same oracle. All four algorithms share the FES accounting described in Section 3, except PSO, which evaluates f inside MATLAB without the XML-RPC subprocess.

8.2 Experimental scale

The largest deliberate deviation is **experimental scale**. The initial report targeted $B \in \{2000, 5000\}$ and at least 30 independent seeds per configuration to support strong inferential claims. Final runs used $B \in \{50, 100, 200\}$ and five seeds because a full $30 \times \text{grid} \times \text{four algorithms}$ would require prohibitive wall-clock time once each

fitness call includes DLQR design and 0.3s horizon simulation (Section 5.4). This directly affects Section 5: Holm-corrected tests on tuned configurations are reported honestly as *partially* significant (Section 5.2), and conclusions in Section 7 are phrased accordingly. Increasing B and seed count remains the highest-priority follow-up.

Hyperparameter grids were retained in spirit (coarse Cartesian products in `experiments_config.txt`) but with smaller ES/ λ sets and PSO particle counts $\{5, 15, 30\}$ aligned to low budgets. Tuning of representative configs uses $B = 50$ rather than the initially suggested $B = 1000$.

8.3 Software engineering changes

Several components were not described in the initial PDF but were required to obtain the final dataset:

- `run_experiments.py` — batch runner with `ProcessPoolExecutor`, progress JSON, CSV summaries, and optional trajectory export;
- **PSO repair** — extension of `PSOMatlabConfig` and `main.m` to pass c_1 , c_2 , ρ_{rang} , and seeds; removal of `parfor` pool startup that broke log parsing; return of `best_fitness` to Python;
- `run_pso_only.py` and `merge_results.py` — auxiliary scripts added when early PSO batches failed or needed replacement in combined tables;
- `plot_results.py` / `generate_report_plots.py` and `statistical_tests.py` / `run_report_statistics.py` — reproducible figures and Holm-corrected tests documented in Section 4.

Bug fixes to `compute_fitness` (robust XML-RPC parsing), ES mutation (NumPy addition), and σ clipping were also necessary for stable experiments but were not part of the initial written plan.

8.4 Analysis and reporting

The initial document mentioned **THD and overshoot** analysis for the best controllers. That step was deferred: the simulator cost (3) already aggregates tracking and control effort, and extracting additional waveforms would require a separate post-processing pipeline not implemented in time. Reporting therefore rests on $f(\mathbf{p})$ and runtime, not on frequency-domain or step-response criteria.

Statistical methodology followed the proposal (Mann–Whitney, Holm, Wilcoxon, Friedman) but results must be read with the reduced n . We explicitly avoid treating pooled grid-wide tests as confirmatory (Section 5.2).

The Springer Nature report template and this deviations section were added for the course deliverable; the initial two-page PDF did not include them.

8.5 What remained aligned

Despite the deviations above, the project still:

- optimizes the same VSI-DLQR simulator and decision vector $\mathbf{p} \in [-10, 10]^4$;

- compares evolution strategies, differential evolution, and PSO under explicit FES budgets;
- stores seeds, hyperparameters, fitness, and runtime in CSV for reproduction;
- discusses limitations and does not overstate statistical certainty where $n = 5$ is insufficient.

In short, the final study is a *scaled-down but methodologically stricter* realization of the initial plan: broader algorithm coverage, stronger software reproducibility, more cautious statistical wording, and smaller budgets/seeds than originally announced.

Code availability

Source code, experiment configuration, and reproduction scripts are available at <https://github.com/AntoniKingston/VSI-Circuit-Optimization-Utilizing-Evolutionary-Strategies>. The repository README.md describes the full workflow from simulator startup to report figures. The course submission may additionally include a git bundle; key entry points are `run_experiments.py`, `generate_report_plots.py`, `run_report_statistics.py`, and the MATLAB/Python sources under `src/` and `algorithms.py`.

References

- [1] Brest, J., Greiner, S., Bošković, B., Mernik, M., Zumer, V.: Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems. *IEEE Transactions on Evolutionary Computation* **10**(6), 646–657 (2006) <https://doi.org/10.1109/TEVC.2006.880136>
- [2] Hansen, N., Ostermeier, A.: Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation* **9**(2), 159–195 (2001) <https://doi.org/10.1162/106365601750190398>